A4 — Spatial Threshold Graphs

Algorithms and Data Structures | ISO C11 | Submit a4.c in Canvas

Problem

Given **n ≥ 1** finite 3D points and a finite radius **r ≥ 0**, form an undirected graph with one vertex per input record. Distinct records remain distinct even if their coordinates are identical. There is one edge for each unordered pair at Euclidean distance **≤ r**. Return the number of edges and whether the graph is connected (1 or 0).

Public API — use the separately supplied a4_api.h unchanged

```
KDTree *kd_build(const Point *points, size_t n);
size_t kd_cube_query(const KDTree *tree, Point center, double r,
                     size_t min_index, size_t *indices);
void kd_free(KDTree *tree);
GraphStats graph_stats(const Point *points, size_t n, double r);
```

Independent work — k-d tree and all its helpers

Implement **kd_build**, **kd_cube_query**, **kd_free**, and supporting k-d-tree code independently. Build a genuine 3D k-d tree preserving each point's original zero-based index, including duplicate coordinates. Every subtree must store its **maximum original input index**. Leave the caller's point array unchanged and free only memory owned by the tree.

The cube query returns each original index **j ≥ min_index** exactly once when $|x_j - \text{center.x}| \le r$, $|y_j - \text{center.y}| \le r$, and $|z_j - \text{center.z}| \le r$. Use both **spatial subtree pruning** and **subtree-maximum-index pruning** during search, not an unrestricted spatial search followed by index filtering. Include cube boundaries; equality must not wrongly eliminate a subtree.

The caller provides space for **n** indices; output order is arbitrary. Return the count written, or zero when none qualify or **min_index ≥ n**. Query centers need not occur in the input.

AI assistance permitted — graph analysis only

graph_stats must build the tree, for each input point **i** call **kd_cube_query** with cube half-width **r** and **min_index = i + 1**, then use squared Euclidean distance **≤ r²** to filter cube candidates. Count each edge once; determine connectivity using any suitable method; release allocated memory. Repeated calls must work. For **n = 1**, report zero edges and connected = 1.

Submission and AI-use restrictions

Submit **exactly one source file, a4.c**, containing all four functions and any helpers. Begin with **#include "a4_api.h"**. Do not modify the header or define **main()**; the grader supplies both header and main. No input/output, command-line interface, or display is required.

AI may assist only with the graph-analysis component. All k-d-tree code (construction, querying, pruning, freeing, structures, and helpers) must be your own work: do not use AI to write, debug, inspect, rewrite, restructure, or refactor it. **Compose your AI prompts independently.** Do not supply the assignment specification **or any part of it** to an AI system for any purpose: do not upload, paste, transcribe, screenshot, quote, or otherwise communicate the PDF or its contents. **Sole exception:** you may give AI the separately distributed **a4_api.h** to integrate permitted graph-analysis code.